When the Interval Is Smaller Than the Instrument: Two Ways Streaming Latency Benchmarks Fail on Sub-Millisecond Paths

Gustavo Pedro Ricou

***Abstract*—Streaming brokers are compared on sub-millisecond paths with instruments whose timescales exceed the intervals they report. One ratio governs what follows: an instrument timescale over the interval measured. Two consequences of that ratio exceeding one are invisible in the output. First, a measured latency can come out negative, because both stamps are written by threads that must first be scheduled, and the difference between those two delays can outlast the delivery timed. A sign check rejected $1{,}321$ of $2{,}266$ runs. Across $738{,}730$ events the acknowledgment-referenced span inverts $62{,}264$ times while the send-referenced span never does, on one clock by construction, so the cause is scheduling and not synchronization; real-time priority at fixed utilization collapses it 7–$80\times$. Second, the most widely used cross-broker benchmark admits an *end-to-end* sample only when a millisecond-quantized difference is positive and counts none of what it drops, while measuring its same-process span with a nanosecond clock, unfiltered: over 71 embedded-mode cells reporting a median on the millisecond grid it computed that median from 0.36% to 100% of the samples taken, at a retained fraction fixed by the send-interval-to-quantum ratio. The remedy is one line: dither the send instant and publish that retained fraction.**

***Index Terms*—distributed systems; measurement techniques; Measurement, evaluation, modeling, simulation of multiple-processor systems; message passing; performance attributes; scheduling and task partitioning.**

I. Introduction

STATED plainly, this paper reports two ways a latency benchmark can be wrong about its own measurements, and one number that governs both.

A benchmark times a message by reading a clock when it is sent and again when it arrives, then subtracting. Nothing about the message takes those readings: two *threads* do, each of which must first be given a core, and each of which waits for one on its own account. An *interval*, here and throughout, is that one span from send to arrival — not a clock period and not a sampling window — and on a busy machine either wait can outlast it. When the two waits *differ* by more than the interval, the subtraction goes negative: nothing arrived before it was sent, the two readings were simply taken in the wrong order. Separately, the most widely used cross-broker benchmark for comparing brokers stamps its cross-process span in whole milliseconds and keeps only positive differences, so on a path faster than a millisecond it deletes samples at a rate fixed by arithmetic rather than chance. Neither failure appears in the output, and both are caught by checks that cost nothing.

G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie.

The two failures share one number. Write T_{true} for the interval being measured and put beside it whichever timescale the instrument contributes, the scheduling stall before a stamp is written or the resolution of the stamp itself. When that instrument timescale exceeds T_{true}, the measurement stops describing the system and starts describing the instrument. Sub-millisecond broker paths are where that ratio crosses one for tooling in current use.

That a measurement degrades as the interval shrinks towards zero is not the claim, and would not be worth making: every instrument has such a limit, as sampling has its Nyquist frequency. The claim is about where the limit falls, and it does not fall where the datasheet says. The stamp is one millisecond, but the timescale that binds is the scheduler's: the run-queue stall distribution's last mode sits at 3 ms (Section V-D), three times the quantum and nothing a specification discloses. A practitioner reasoning that a 0.5 ms path and a 1 ms stamp are the same order of magnitude, so the measurement should be roughly sound, is wrong by a factor they had no way to look up — and the instrument gives no sign when the line has been crossed.

A. Contributions

1) **A deletion law for benchmark samples, established by manipulation** (Section IV). The OpenMessaging Benchmark admits an end-to-end sample only when a millisecond-quantized difference is positive, with no counter for what it discards. Across the 71 cells that report a median on the grid it keeps between 0.36% and 100% of what it takes; over the full ledger of 223 runs the floor is 0.0044%. Retention follows arithmetic the operator never sees: with send interval over quantum equal to p/q in lowest terms, retention can take only the $q{+}1$ values $0, 1/q, \ldots, 1$, and a run lands on one of the two that bracket T_{true}/τ. Fourteen arms across seven denominators behave as positioned, 9 of 10 powered arms reject a continuum null after correction, and a pre-registered payload manipulation moves an arm onto a grid vertex and off it.
2) **The negative-span mechanism, established by manipulation on one clock** (Section V). A span comes out negative when the two stamping delays *differ* by more than the interval being measured — delaying both stamps together inverts nothing — so $\Pr[\text{span} < 0] = \Pr[\Delta < -T_{\text{true}}]$ for Δ the difference between

them (Equation 2). Real-time priority on the stamping threads at fixed utilization collapses the rate 7–80× over 8 matched pairs; identical utilization reached by two core geometries differs 2.07×; lengthening the true transport 77× *lowers* the rate; an unfitted kernel trace predicts the observed rate to within a third. The span that inverts is referenced to an acknowledgment, and the send-referenced span over the same 738,730 events never inverts, locating the fault in the reference stamp. Two brokers sharing neither client nor protocol invert at 8.67% and 8.19% on one clock: the fault follows the span, not the system under test.

3) **An audit, and the reporting rules that follow** (Sections III-B and VI-B). A sign check applied uniformly to all 2,266 runs rejected 1,321 of them, including every run behind our own first result. The practice is older than this paper, and each part of it is established: physically impossible values are grounds for discarding a trace [1], a standards body already voids a whole run on a clock criterion [2], and SPEC and TPC name *verifiability*, publishing more than the metric needs so inconsistency shows, among a benchmark's quality criteria [3]. What we add is their conjunction — a physical-impossibility criterion, applied per run, with the campaign's rejection rate published as a headline quantity rather than a footnote — and the observation that the retention rate and the sign of each discard belong beside every reported latency.

B. Chronology, and what was withdrawn

We set out to compare two brokers on a live sports feed. The audit of Section III-B rejected our own first answer — a complete, statistically significant result set — and a later audit refuted a claim we had already published about the benchmark. What follows is what survived; Supplement S35 has the order of discovery, the withdrawn claims and the failed pre-registrations, an audit hiding its own failures being worth less than one that does not.

II. Background and Related Work

The claim that measurement can dominate a benchmark is not new; what is missing is the practice of checking for it run by run.

A. Streaming benchmarks

Broker comparisons appear continuously, in peer-reviewed venues and in the gray literature practitioners read; Apache Kafka and Redis Streams are the pair they most often name, and the OpenMessaging Benchmark [4] is the shared instrument; Supplement S52.3 sizes that literature. That experimental setup can dominate a result is established [5], and how to report one is a settled literature we follow [6], [7]. Most of those comparisons report a median above the quantum, where the law of Section IV does not bind — the scope of our claim rather than a limit on its reach.

The instruments are few and shared, so a defect in one is not one project's problem: the audit of Section IV-D reads seven tools at source and finds five disposing of inverted samples without counting them. Nor is the architecture that produces those negative spans a blunder to be designed away. Every mainstream client sends asynchronously and signals completion on a callback thread, because a producer that blocked for each acknowledgment could not generate load at all; the send is stamped on the calling thread and the reference on another, and no design that keeps the asynchrony has one thread to stamp with. What a benchmark can drop is the habit of subtracting two such stamps without ever checking the sign of the result.

The stream-processing benchmark literature has already thought carefully about where to stand a clock. Karimov et al. [8] separate event-time from processing-time latency and keep the driver off the machines under test; Van Dongen and Van den Poel [9] use a single broker and its append timestamps "to ensure correct latency measurements", which is our one-clock rule arrived at independently. Neither work asks what a millisecond stamp does to a sub-millisecond path, or checks the sample the instrument keeps. Those two questions are this paper.

B. Cross-process time measurement

Lamport [10] established that a distributed system has no single physical time, only a causal partial order. That order decides this paper's central span: an acknowledgment at the producer and a record at the consumer are two branches from the broker's append, so neither precedes the other. Agreeing clocks is a well-known engineering problem [11] and instruments are built to beat it, calibrating cross-node counters to tens of microseconds [12], stamping in the NIC [13] or the switch dataplane [14], and measuring the timer's own accuracy [15].

One prior result bounds our claim closely. Villain et al. [16], profiling FreeBSD against a DAG hardware reference, found the outgoing socket timestamp "does not respect causality" — packets leave before the stamp meant to mark their departure is written — under every stress pattern including none, and attributed host latency to interrupt processing, scheduling and context switching. That is Mode A's physical primitive, published in 2012 and measured more precisely than we measure it; we add its consequence for a two-process measurement, a law relating the negative-span rate to the interval measured (Equation 3), and manipulations separating scheduling from its rivals. The audit half has its own ancestor: Paxson [17] treated physically impossible transit times as instrument failure and published the proportion of traces affected. Lancet is the nearest existing gate, and reports an inconclusive verdict with the results attached where we decline to publish at all; neither it nor anything else we found reports the fraction of runs a campaign lost.

Production tracing reached the same answer without a mechanism: skew adjusters that rewrite a span's timestamps have been called harmful and are off by default (Supplement S52.1). Where the displacement is a late stamp rather than a skewed clock, correcting it moves evidence rather than error — and nobody had measured how much.

Concurrent work reaches the same observable by another route. In a 2026 preprint, Sharma et al. [18] count negative timing spans in distributed inference systems and propose a `causality_health` check; injecting skew, they see no violations up to 3 ms and clear ones by 5 ms, and read that as a skew threshold. We take their term, and dispute the reading rather than their measurements. They hold that "queueing alone cannot produce negative timing spans"; under the queue they model, backlog at a service stage, we agree, but a *second* queue decides ours — the run queue the stamping thread waits in for a CPU. Eliminating backlog therefore does not leave skew as what remains: we see negative-span rates near 23% at 88% utilization on one host, with an inter-host offset of 0.067 ms where two are used, and move that rate fiftyfold without touching a clock, so a threshold in milliseconds of skew does not transfer to a loaded machine (Supplement S50). The same attribution has been made about the benchmark we audit: asked why publish latency could exceed end-to-end latency, a maintainer cited inter-node clock skew of "~100ms" [19] and, told that both processes ran on one node, called the measurement reliable. That is the arm our audit rejects most often.

C. Resolution and discarded samples

That a timer coarser than the interval invalidates it is textbook [15]. Metrology has long carried both our failure modes in one uncertainty budget: the NIST stopwatch guide budgets reaction time beside display resolution, and notes that at short intervals the resolution becomes significant against the instrument's rated accuracy [20]. We claim neither the pairing nor its novelty. That periodic sampling commensurate with a periodic phenomenon sees only part of it is stated in the IPPM framework [21], and RFC 3432 makes a randomized start mandatory for periodic streams [22]. Coordinated omission [23] is the mirror image — there the instrument fails to record samples that would have been slow, here it deletes samples that were fast — and Tene's point, that omission correlated with the value measured skews the result, covers both. The precondition has been seen before in another benchmark: YCSB reported sub-millisecond operations as zero milliseconds until it adopted microsecond histograms [24], and it kept those samples rather than deleting them. Millsap and Holt state the same arithmetic and draw the opposite conclusion from it [25]: an event shorter than the resolution measures one tick or zero according to whether it crosses a boundary — our Equation 4 — and coarse timing survives in aggregate *because* the two signed errors cancel. A guard admitting only positive samples removes one of the signs, and with it the cancellation the defence depends on (Supplement S41).

The mathematics of the interaction is older still, and it is not ours. Counter metrology settled it for time-interval averaging half a century ago, and Hewlett-Packard's Application Note 162-1 sets out all of it [26]: its retention identity is our Eq. 4, its *class* of a repetition rate our denominator q under the same co-prime condition, and its ceiling on any gain in resolution our Eq. 5. It even lists the symptom — a counter reading that appears to hang on a round number — and prescribes the cure we arrived at independently, phase jitter on the repetition rate (Supplement S51 maps the correspondence line by line).

What is new is the sign of the operation. HP *keeps* both readings, and the retained fraction *is* the estimator: averaging is unbiased precisely because the $Q{+}1$ readings are counted alongside the Q readings. The benchmark discards that population. Our contribution is therefore not the law — we derived it, then found it in a counter manual — but its consequence under deletion: a harness released in 2018 throws away the quantity a 1970 counter measured with.

a) Sources of scheduling delay: A runnable thread does not always run, and work-conservation violations are documented for CFS-era schedulers [27], ours being EEVDF-era. Queueing gives the leading-order account: utilization alone does not fix a waiting time, so the geometry contrast of Table II refutes a single-parameter law rather than queueing itself. A 2026 preprint by Chandrasekar and Kramberger [28] applies such a law to benchmark bias; we refute it on the load axis (Section V) and it reaches our inflation half independently. What a positive number cannot show — the negative span, and the sign channel that makes the displacement legible — is what remains ours (Supplement S52.4).

III. Method

Everything below is measured on one clock unless the text says otherwise, and the spans we audit are named explicitly, because which two stamps are subtracted is the whole question. A span whose difference comes out below zero we call a *negative span*. The term is Sharma et al.'s [18], taken deliberately: we keep it for the observable and decline the causal reading they give it, because what a negative span establishes is that one of two stamps is untrustworthy, not which. *Span* is used in the tracing sense, a pair of stamps bracketing an interval, and nothing here is specific to a tracing system; nor is this RFC 1242's negative latency, a cut-through forwarding artifact [29]. *Retention* is the fraction of the samples an instrument takes that survive into what it reports, not Kafka's log-retention setting.

A. Testbeds, and what the measurement proxies

Every finding comes from four Oracle Cloud `VM.Standard.E5.Flex` instances (three brokers, one driver), Ubuntu 22.04 on kernel `6.8.0-1057-oracle` (EEVDF scheduler), CPython 3.10, a private network, client libraries pinned identically, and `chrony`-disciplined clocks logged every minute. One early phase perturbed the measurement enough to be unusable and is excluded from every result here; the campaigns that remain measure utilization with no core pinning (Supplement S35). A second testbed, a workstation running the brokers under containers, produced our withdrawn first result and appears only as an audit outcome.

a) The measurement at a glance: The harness runs producer and consumer as separate *processes on one host*, both stamping the wall clock (`time.time_ns()`, CLOCK_REALTIME), not a monotonic counter or the TSC, since a cross-process span needs a shared epoch that no per-core cycle counter provides. The cost is worth naming: only CLOCK_MONOTONIC guarantees consecutive reads do not go backwards, so we forgo an exclusion the clock could have supplied and establish it from data instead (Section V). Nor did we record `current_clocksource`; the probe now does, too late for the runs reported here. The workload is a public football event feed (StatsBomb open data, CC BY-NC-SA 4.0) replayed against Kafka and Redis Streams. The send schedule is measured rather than assumed: pacer jitter is 67–69 μs at p90, and the achieved rate is recorded per run against elapsed wall time — the self-validation SPEC and TPC ask of a benchmark [3], applied to the load generator rather than the system under test. The measured inter-host offset, where two hosts are used, is 0.067 ms. The primary measure is **Time-to-Insight** (TTI), from scheduled emission to availability at the consumer:

$$\mathrm{TTI} = \underbrace{(t_{\mathrm{send}} - t_{\mathrm{sched}})}_{\text{scheduling lag}} + \underbrace{(t_{\mathrm{recv}} - t_{\mathrm{ack}})}_{\text{transport proxy}} + \underbrace{\text{residual}}_{\text{broker-internal}} . \quad (1)$$

The residual is the broker's own append-to-deliver interval, which no stamp of ours brackets; it is carried because TTI is a total, and named because an unnamed span is not a latency.

b) One component is a proxy, not a chain: This distinction is load-bearing. The transport term of Equation 1 subtracts the producer's receipt of the broker's acknowledgment from the consumer's receipt of the record. The broker's append precedes both of those events, but neither of them precedes the other: they are two branches of one cause. The acknowledgment stamp is therefore a *late, producer-side observation* of a broker-side event, and when the acknowledgment branch runs slower than the delivery branch the difference is negative without anything physically impossible having occurred. By contrast $t_{\mathrm{recv}} - t_{\mathrm{send}}$ and TTI itself are genuine chains: the producer sends, the broker appends, the consumer receives. Section V-A reports all four spans, and they behave completely differently.

B. The consistency check

We fix the rule before the final campaign and apply it to every run, including those whose results looked reasonable. **A run is rejected if more than 1% of its events carry a negative value in any latency component, or if the median of any component is negative.** A condition is usable only if all of its runs survive; where partly rejected we report the retention rate. The one-per-cent figure is a threshold, not a discovery: the supplement sweeps it from 0 to 20% and no condition behind our first result is usable anywhere in that range, so the withdrawal does not turn on where the line was drawn.

The justification is not that a negative is impossible, because for the transport proxy it is not. It is that a negative proves the reference stamp cannot serve as the origin of that event's interval. A run whose reference is unusable on more than one event in a hundred cannot report a latency, whatever the cause. The check is a *necessary* condition, not a validation procedure: a corpus that fails is unusable, and one that passes has cleared the cheapest available bar and nothing more. We use the sign as a gate rather than as an estimator, although the same constraint supports estimation, since offset and skew can be recovered from a delay envelope [17]. That was deliberate: an estimator corrects a record, a gate declines to publish from a run whose instrument demonstrably failed, and after this check destroyed our own first result we preferred declining.

C. A model of the failure

A negative span is not a clock failure but an ordering failure between two stamping delays, and the model that says so is two equations long. Every timestamp is taken some interval after the physical event it claims to mark, because the thread that reads the clock must first be scheduled. This is not the cost of the clock call, which is small and separately measurable [15]; it is the delay before the call runs at all. Writing δ for those stamping delays,

$$T_{\mathrm{measured}} = T_{\mathrm{true}} + \Delta, \qquad \Delta \equiv \delta_{\mathrm{recv}} - \delta_{\mathrm{ack}}, \quad (2)$$

so Equation 2's difference inverts exactly when $\Delta < -T_{\mathrm{true}}$, giving

$$\Pr[\text{span} < 0] = F_{\Delta}(-T_{\mathrm{true}}). \quad (3)$$

Figure 1(a) names the four stamps; Supplement S45 maps them onto the span each metric covers.

Equation 3 says the most useful thing in this paper (Fig. 1b). **Negative span probability is a property of the quantity being measured, not only of the machine**: a fixed distribution of Δ overlaps a small T_{true} almost entirely and a large one hardly at all, which is why our network-delay arm, transport in tens of seconds, passes every run while same-hardware arms measuring one millisecond fail wholesale.

D. Statistics

Every proportion below is a count over a stated denominator with a Wilson score 95% interval, Wilson rather than Wald because at the real-time arms' rates the normal approximation puts the lower bound below zero. Holm correction is applied within a named family and the corrected value is the one the tables display. Tail indices are estimated on the traced histogram by grouped maximum likelihood and by an exceedance ratio, never by a slope through survival points, because the buckets are interval-censored; the grouped estimator on log-spaced bins is Virkar and Clauset's, which is the Hill estimator for binned data, and we use their bootstrap for goodness of fit [30]. The remaining estimators — pooled-variance z, Hodges–Lehmann, TOST, the permutation null, the bootstrap — are inventoried in Supplement S49. All interval arithmetic is recomputed from the committed artifacts at build time by `scripts/stat_intervals.py` and `scripts/tail_index_traced.py`, and the tables reporting it are generated rather than transcribed.

IV. Mode B: A Benchmark That Deletes Its Own Samples

The cross-broker benchmark we audit computes its reported latency distribution from a subset of the samples it collected,

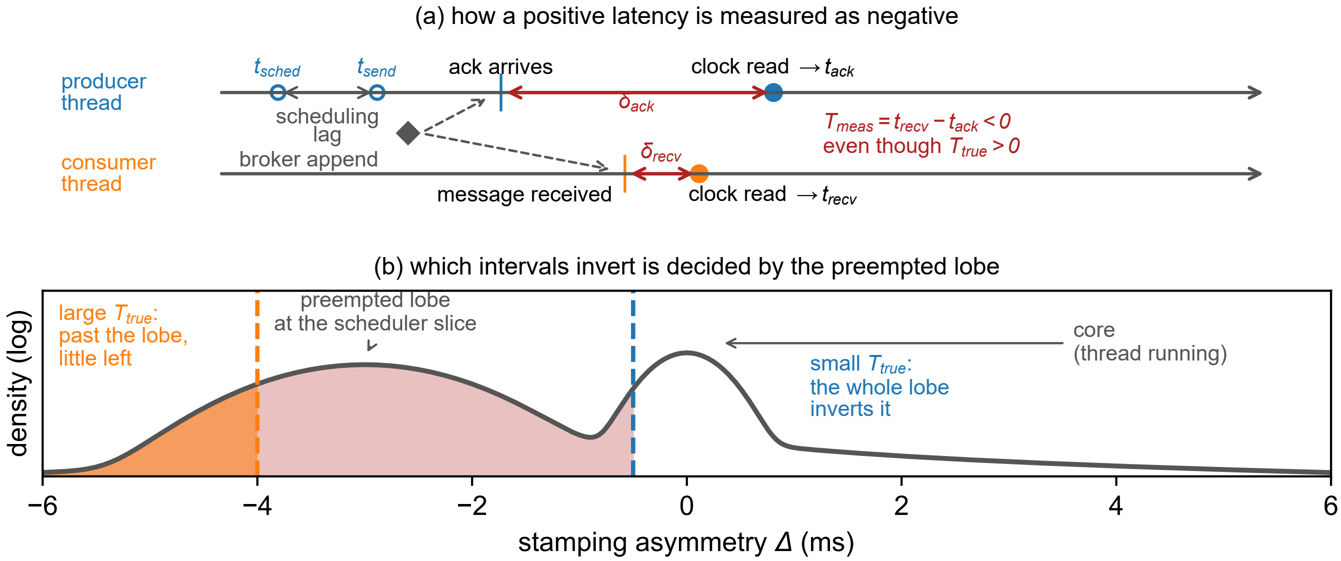

Fig. 1. **A late stamp, not an early record.** (a) How a positive latency is measured as negative, on one clock: each side stamps some interval after its event, and when the producer's stamping delay exceeds the true delivery plus the consumer's delay, the difference inverts (Section III-A). The producer's own stamps, t_{sched} and t_{send}, do precede the acknowledgment, and the span referenced to them never inverts (Table I). (b) Why the check bites hardest on small effects: the stamping asymmetry Δ has a narrow core when the stamping thread is running and a lobe at -3 ms, the scheduler's base slice, when it is not, so a small T_{true} is inverted by the whole lobe and a large one only by what lies beyond it.

does not record how large that subset was, and the size of the subset is determined by the send rate. Everything measured below is the OpenMessaging Benchmark in embedded mode, where driver and workers run in one JVM; its distributed mode is discussed in Section VI-D.

A. The guard, and its origin

In `LocalWorker.java`:294 the end-to-end latency is `now - publishTimestamp`, where `now` is read in the consumer and `publishTimestamp` is Kafka's producer-set `CreateTime`, itself a millisecond value. That difference is then promoted, `TimeUnit.MILLISECONDS.toMicros(now - publishTimestamp)`, so the guard in `WorkerStats.java`:95 — `if (endToEndLatencyMicros > 0)` — tests a variable named for microseconds that can only ever hold integer multiples of a thousand of them. The message is counted as received a few lines earlier, but a non-positive latency is dropped and no counter records how many.

The guard's origin is documented and its reasoning is sound. It was added in 2018 to stop negative values reaching the histogram, on the ground that end-to-end latency "is calculated based on time difference of `System.currentTimeMillis()` on two different machines" and "can be negative" [31]; the recorder is an `HdrHistogram Recorder`, which rejects negative values outright. Filtering for positives is the reasonable local fix, defensive rather than evasive. Its non-local consequences are two, neither considered at the time. First, `> 0` also deletes every sample that computes to exactly zero, and because `System.currentTimeMillis()` has millisecond resolution, any delivery faster than one millisecond does compute to zero; on co-located paths, where our own transport measures 0.1–0.5 ms, that is most of them. Second, the one observation which would prove the instrument had failed is the exact observation the guard removes. Section IV-D finds the same disposal, uncounted, in four independent tools.

The guard's scope is the most interesting thing about it. It is on the end-to-end path, which stamps in milliseconds; the same harness measures its publish span with a nanosecond clock and records every sample unconditionally, needing no guard because that span is taken inside one process. The coarse clock and the filter both landed on the one span that crosses processes.

a) The consequence, measured: Of the 75 instrumented cells whose own summary we captured, 71 report a median of 1.0 or 2.0 ms, and across those the benchmark computes its reported distribution from between $\mathbf{0.36}\%$ and $\mathbf{100}\%$ of the samples it takes. Two replicates of one cell — 200 B payload at 500 msg/s, same host — kept 431 and 120,434 samples, a 279-fold difference, and both reported a median of 1.0 ms with nothing in the output to distinguish them. Three uninstrumented runs print p50, p95, p99 and max all equal to 1.0 to three decimal places. The full ledger holds 223 instrumented runs, 11,532,492 discarded samples and 41,403 negatives, and over it the retained fraction falls as low as 0.0044% — four samples kept of ninety thousand. Figure 2 puts the two quantities on one pair of axes: the printed median is pinned at the quantum while retention moves across two and a half orders of magnitude, and the only cells that escape are the four whose payload is large enough that the quantum stops binding.

That discarded fraction is the one time-interval averaging measures with [26]: reporting a distribution conditioned on it,

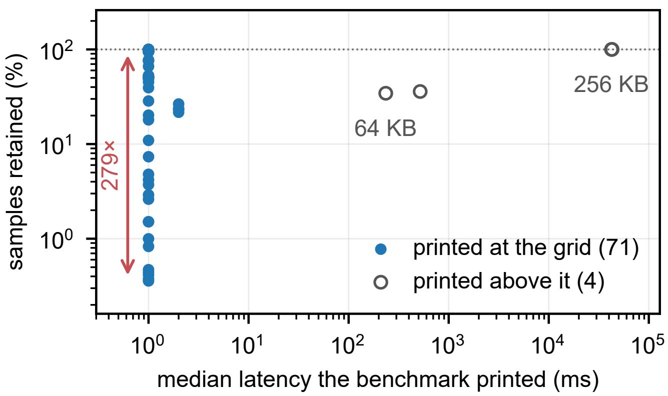

Fig. 2. **What the benchmark prints against what it kept in order to print it**, over every instrumented cell. At the quantum the printed median carries no information about retention: 71 cells report 1.0 or 2.0 ms while the retained fraction spans 279×. Away from the quantum the harness behaves, which is the point — the failure is confined to intervals shorter than the instrument.

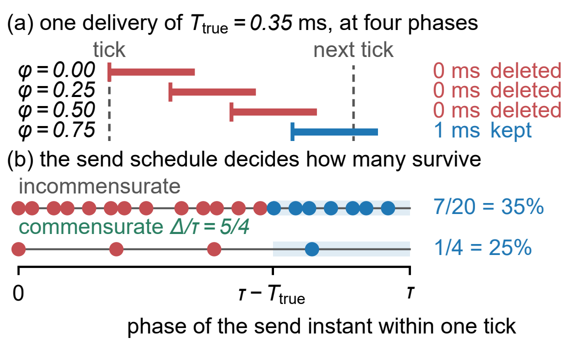

Fig. 3. **Why a sample disappears, and why the survivors are not a sample.** (a) One delivery of $T_{\text{true}} = 0.35$ ms at four phases of one tick. Three finish inside it, so both stamps round alike, the difference is exactly zero, and the guard removes them; the fourth crosses the boundary and is recorded as one whole tick — which is why the corpus prints 1.0 and 2.0 ms and nothing between. (b) A delivery survives from phase $\tau - T_{\text{true}}$ onward — the shaded band — so uniform phases retain T_{true}/τ (Equation 4); a commensurate producer visits only q and retains a count out of q, one of the values Equation 5 allows. T_{true} here is illustrative and off-vertex by choice: at 0.5 ms the two answers coincide.

without reporting it, keeps the residue and throws away the estimator.

B. Phase, and the grid it produces

Publish latency across these cells takes only two values, 0.3 and 0.4 ms, while retention varies 279-fold. A two-valued predictor cannot account for a range like that whatever its correlation, and the correlation it does have (+0.31 over 71 cells) carries the sign Equation 4 predicts. What moves retention is where the producer's send instants fall against the millisecond grid. A sample survives when its delivery crosses a tick boundary, which for a delivery shorter than one tick happens with probability T_{true}/τ (Figure 3):

$$\text{retention} = \min\left(1, \frac{T_{\text{true}}}{\tau}\right) \qquad \text{(uniform phases).} \tag{4}$$

Both incommensurate arms sit near 50%, implying $T_{\text{true}} \approx 0.5$ ms at $\tau = 1$ ms, which agrees with our own unquantized transport measurement.

a) Commensurability is not binary: Phases are not uniform when the send interval is commensurate with the tick. Write the send interval over the tick in lowest terms as $\Delta/\tau = p/q$ with $\gcd(p, q) = 1$: a producer visits exactly q distinct phases, retention is a count out of q and can take only the $q+1$ values $0, 1/q, \dots, 1$, and a run lands on one of the two bracketing T_{true}/τ. Coarser grids are less reproducible, $q = 1$ being the all-or-nothing corner and an incommensurate rate effectively continuous (Supplement S23). The replicate spread follows:

$$\text{spread} = \begin{cases} 100/q & \text{when } T_{\text{true}}/\tau \text{ sits mid-cell,} \\ 0 & \text{when } T_{\text{true}}/\tau \text{ sits on a grid point.} \end{cases} \tag{5}$$

Supplement S23 reports every arm. A round number is the worst possible choice: 500 msg/s is exactly 2.000 ms, $q = 1$, and its four replicates retain 0.47, 1.51, 18.02 and 99.99 per cent, where an incommensurate 457 msg/s holds to 2.1 points. The exactness matters: 889 msg/s sits 0.00014 from 9/8 and behaves as fully continuous.

C. The grid as inference

A table of spreads is not a test, so we state one. For each arm let D be the mean distance from its replicates to the nearest vertex of the q-grid, normalized by the cell half-width, and let the null be a permutation of replicate positions within the arm, which is what a continuum implies. Supplement S54 gives the construction and Supplement S46 every arm with $n \geq 5$, printing the corrected value rather than the raw one. 9 of the 10 powered arms reject the continuum null at $\alpha = 0.05$ after Holm correction within the family of 12. The tenth, 900 msg/s, rejects before correction ($p = 0.044$) and not after ($p = 0.131$); we report it as unresolved rather than as support. The remaining 2 arms, 400 and 800 msg/s, have no power by construction: when T_{true}/τ sits on a vertex the two predictions coincide and no test can separate them; a model that claimed separation everywhere would be the more suspicious one. Figure 4 shows why the set matters more than any one arm: every powered arm falls below the diagonal and the one that does not reject is drawn as such. The two arms above it are the ones no test can separate, and we claim nothing from that position — where the predictions coincide the distance is noise, and which side of the line noise lands on is not evidence either way.

a) A pre-registered confirmation: Payload moves T_{true}, hence the grid position, and Equation 5 dictates the arm's class with nothing else changed. The prediction was registered before the runs, and it came true (Fig. 5): spread collapses to 13.6 at 32 KB, three replicates within 0.06 of each other, pinned near the 2/3 vertex, and re-opens to 26.7 at 64 KB, the flat–full boundary crossed in both directions by one manipulated variable. One registered detail failed: the 32 KB pin sits at 69.2, not the predicted 66.67.

A larger campaign registered six predictions with their falsifiers before running 63 cells; three fired, and what survives is per-send jitter rather than accumulation (Supplement S55).

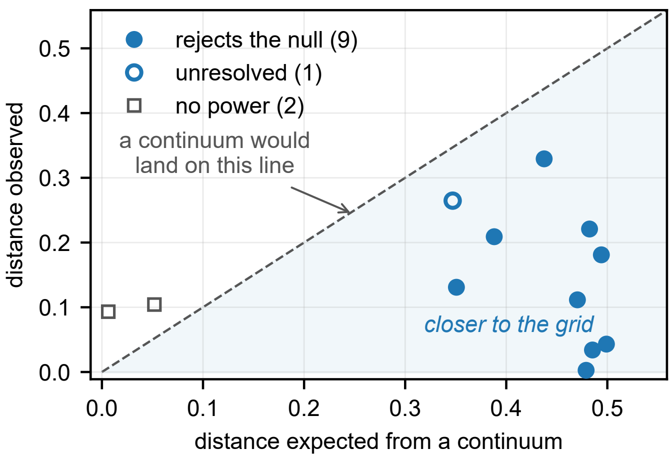

Fig. 4. **Grid membership, as a set.** Each arm's observed mean distance to the nearest grid vertex against the distance a continuum would give. Points below the diagonal sit closer to the grid than chance allows. Filled circles reject the continuum null after correction; the open circle is the one powered arm that does not, drawn apart so the figure claims no more than the test does. The 2 squares are the arms where the two hypotheses coincide and no test has power; there the normalized distance is replicate noise and *its sign carries nothing*, so their falling above the diagonal is not a prediction met. Per-arm corrected p-values are in Supplement S46.

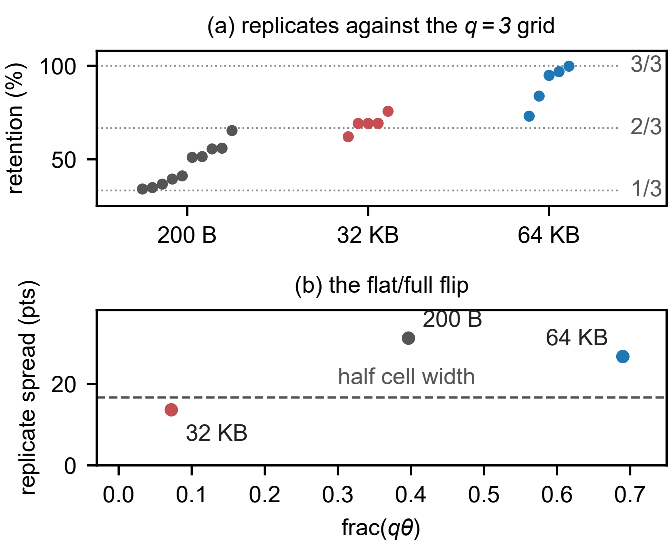

Fig. 5. **The campaign's confirmed prediction.** (a) Retention replicates at 300 msg/s ($q = 3$) by payload, against the thirds grid: 200 B straddles both branches, 32 KB pins near the $2/3$ vertex, 64 KB re-opens to the upper cell. (b) Replicate spread against $\mathrm{frac}(q\theta)$: the flat–full boundary at half the cell width is crossed in both directions by the one manipulated variable.

D. Generality, and the sign channel

To show the law is not an artifact of one JVM, we reproduced the guard in an independent Python harness sharing no code with the benchmark. On one clock it recorded **zero** negative differences across $905{,}040$ samples, as a causal chain requires, while the grid behavior reappeared intact. Across two `chrony`-disciplined hosts retention wandered from 13.4 to 27.0% over four replicates where the same arm on one clock held to 0.8 points: cross-host, the deleted fraction couples to the synchronization state and nothing records either.

Nor is the disposal confined to this harness. We audited seven tools at source across four languages and seven vendors, classifying each finding with the code that classified the benchmark; five of the seven dispose of the sample without counting it, in three classes. Two admit only positive samples, one of them Apache Pulsar's performance client reaching the same design independently [32], which makes this a pattern rather than one project's oversight. Two do something worse and substitute a constant — `fio` returns zero [33] and `btt` one nanosecond [34] — because substitution leaves nothing missing, so retention computed downstream reads 100% and a value never measured enters the distribution. One hands the sample to a recorder that refuses it and does not check the return. Of the remaining two, one counts its discards in a metric whose description names the cause [35] — the proof that the rule of Section VI-B is practical rather than aspirational — and one discards nothing at all, guarding a counter rather than the sample; we report the last because we first read it as a filter and it is not [36]. Supplement S36 gives the registry, tool by tool, with the line each finding sits on.

The class is not confined to benchmarks: in November 2025 the audited broker's own coordinator metrics adopted the same disposal, clamping negative durations to zero with no counter and no log line [37]. Its mechanism is a backward clock step rather than the stamping asymmetry of Section V, so Supplement S36 sets it out there rather than beside Mode A.

Separating the discards by sign is what makes the guard's cost legible. Across all 223 runs the Kafka-driver corpus's $10{,}913{,}263$ discarded samples contain **not one negative**: the most negative value at any load, size or rate is $0\,\mu$s. Those discards are resolution failures, not clock failures — a distinction an unsigned counter cannot make. The same channel then caught the real thing in the Redis-driver replication: $41{,}403$ negatives, every one exactly $-1000\,\mu$s, the minimum the quantum can express, with $41{,}397$ of them in a single run where they were *half of all samples taken*, beside six kept.

That value is not a coincidence. The Redis driver's span is a difference between two millisecond-floored clocks on two hosts, and a sub-tick offset between two floored clocks can produce exactly one negative value, $-1000\,\mu$s, and no other; that is precisely what the corpus contains, while the Kafka-driver span, floored the same way but taken inside one JVM, never inverts at all. Clock discipline was clean at every logged minute, which is the point — this failure needs no misbehaving clock, only two of them and a floor (Supplement S53).

V. Mode A: Why a Latency Comes Out Negative

The negatives in our own corpus are produced by scheduling, and we establish this by moving scheduling while holding everything else fixed.

A. What the audit rejected

The rule of Section III-B rejects 862 of $1{,}382$ runs on the workstation testbed (62.4%) and 459 of 884 on the cloud testbed (51.9%), every run behind our first result among them: on that first testbed 8 of 76 conditions survive. At saturation Kafka's median transport proxy reads -6.4 ms at one hundred connections.

TABLE I
NEGATIVES BY SPAN AND BROKER (5,913 RUNS, 738,730 EVENTS, ONE CLOCK). ONLY THE ACKNOWLEDGMENT-ORIGIN SPAN INVERTS, AND IT DOES SO UNDER BOTH BROKERS: KAFKA ON 8.67% OF EVENTS, REDIS ON 8.19%. THE CAUSAL CHAINS ARE CLEAN UNDER EACH BROKER, SO THEIR PER-BROKER COLUMNS ARE DASHED RATHER THAN REPEAT A ZERO THREE TIMES.

Span	Origin stamp	Kafka	Redis	All
$t_{\mathrm{recv}} - t_{\mathrm{ack}}$	acknowledgment	31,899	30,365	62,264
$t_{\mathrm{recv}} - t_{\mathrm{send}}$	send (chain)	—	—	0
$t_{\mathrm{out}} - t_{\mathrm{send}}$	send (chain)	—	—	0
TTI	emission	—	—	0

Table I separates the components over a wider population than the audit: every cloud run that produced matched events, 5,913 of them, rather than the 884 behind a reported result — a condemned run answers which span inverts as well as a clean one does, so none is excluded. The acknowledgment-referenced proxy is negative on 62,264 of 738,730 events (8.43%; Wilson intervals are under 0.1 points here and omitted hereafter), and 3,976 runs exceed the one-per-cent threshold on it. The send-referenced span and TTI, which are causal chains, are negative on **no event at all**. Nothing in this corpus arrived before it was sent; the reference stamp was late. The rejections are therefore evidence of an unusable reference rather than of impossible physics, which is precisely what the gate is for.

a) Why it looked correct: The rejected result was monotone in concurrency, significant after correction ($p = 9.0 \times 10^{-11}$), and passed every conventional check we knew: replicate counts, effect sizes, corrected significance, sanity plots. The acknowledgment stamp is read only once its waiting thread is rescheduled, so under saturation the consumer has already recorded receipt; the send stamp cannot be late that way, whichever client runs (Supplement S43).

b) What the check cannot catch: A great deal: load-induced inflation that stays positive is physically possible and needs a better instrument. Our own second withdrawal is the example — a twentyfold end-to-end gap that was a per-run start-up cost read as a per-event constant, causally consistent and wrong. Causal consistency is necessary, not sufficient.

B. A rare state, not a wide distribution

Each stamping thread is either running, in which case its stamp is late by a narrow jitter of width σ_c, or preempted and off-CPU. Writing $p(\rho)$ for the probability of the second state and $S(\cdot)$ for the residual's survival function, and taking the consumer's stamp to lie in the core — the leading-order case, since both being preempted at once is the product of two small probabilities,

$$\Pr[\text{span} < 0 \mid T_{\mathrm{true}}] \;=\; p(\rho)\; S(T_{\mathrm{true}}). \tag{6}$$

Load changes how *often* the thread that must record the time is not running, not how wide the ordinary jitter is. Going from idle to the knee multiplies the fitted core width σ_c by 5 and the negative-span rate — the mass of Δ beyond $-T_{\mathrm{true}}$ — by 61. The rate has a ceiling below one: fully saturated it reaches 0.37, so even then most stamps are taken by a thread that was running, or late by less than the interval.

TABLE II
THE MECHANISM, BY MANIPULATION (2,985 MATCHED EVENTS PER CELL). TOP: STAMPING THREADS MOVED TO `SCHED_FIFO` WITH BACKGROUND LOAD UNCHANGED; ACHIEVED UTILIZATION MATCHES TO 0.001 BETWEEN ARMS, SO OCCUPANCY ALONE MOVED. BOTTOM: IDENTICAL UTILIZATION REACHED BY TWO CORE GEOMETRIES ($\rho = 0.7531$ IN ALL FOUR k=6 ARMS); A FUNCTION OF ρ RETURNS ONE VALUE FOR ONE INPUT.

Manipulation	Arm	Rate	95% CI	Factor (z)
Priority, 75%	ordinary	0.1320	0.1203–0.1446	39× (19.8)
	real-time	0.0034	0.0018–0.0062	
Priority, 88%	ordinary	0.3049	0.2886–0.3216	54× (31.9)
	real-time	0.0057	0.0036–0.0091	
Geometry, original	concentrated	0.0824	0.0731–0.0928	2.07× (10.3)
	spread	0.1709	0.1578–0.1848	
Geometry, repl.	concentrated	0.0600	0.0520–0.0691	2.05× (8.4)
	spread	0.1229	0.1117–0.1352	

C. The experiments that settle it

No pair of arms in Fig. 6 overlaps. Real-time priority on the stamping threads moves occupancy while utilization stays fixed (Table II, top). The rate falls by factors of 39 and 54; the Wilson intervals of the two arms are disjoint at both load levels ($z = 19.8$ and $z = 31.9$), and both residual rates land on the scale the model names in advance, $C_0 \approx 0.004$; over all 8 pairs the manipulated arm runs from 0.0017 to 0.0144, so C_0 is where the rate settles and not a bound it respects. *This refutes every account in which the negative-span rate is a function of utilization*, including the queueing form we ourselves adopted: ρ did not change and the rate changed fiftyfold. Across 8 matched pairs spanning 60 to 95% load the factor ranges 7–80×, every pair with disjoint intervals; Supplement S47 reports each one, with the campaign it came from.

Three further manipulations agree. At $k = 6$ both load geometries sit at $\rho = 0.7531$, identical to four decimals, and the negative-span rates differ by 2.07× ($z = 10.3$), reproduced at 2.05× by a full replication. Multi-server queueing expects geometry to matter at fixed ρ, so this will not surprise a queueing-literate reader; what it rules out is the single-parameter form we had adopted and pre-registered, where ρ alone fixes the rate. Payload padding lengthened transport 76.9× while the negative-span rate *fell* 4.1×, monotonically, at a 0.012 utilization spread: the direction Equation 3 predicts and the opposite of a stress account. Finally, a kernel trace on the `sched_wakeup` and `sched_switch` tracepoints [38], filtered to the harness processes and sharing no instrument, estimator or data with the application-level rate, predicts that rate to within a third across 3 arms at two load levels — ratios $0.78, 1.06, 1.32$, no consistent sign, nothing fitted. The probe is not free: 0.272 untraced against 0.231 traced ($z = 3.6$), and the comparison uses the traced arm's own rate, so 0.272 is the machine's (Supplement S43.4).

a) The load-axis predictions, and their failure: The knee sweep placed conditions where the candidate load laws diverge. The M/G/1 form fitted *worse than the mean* ($R^2 = -0.05$ against a fitted exponential's 0.93), and our own registered bracket missed a 1.44× growth by predicting 2.45–3.07×. Both were parameterized in ρ, and ρ is not the variable.

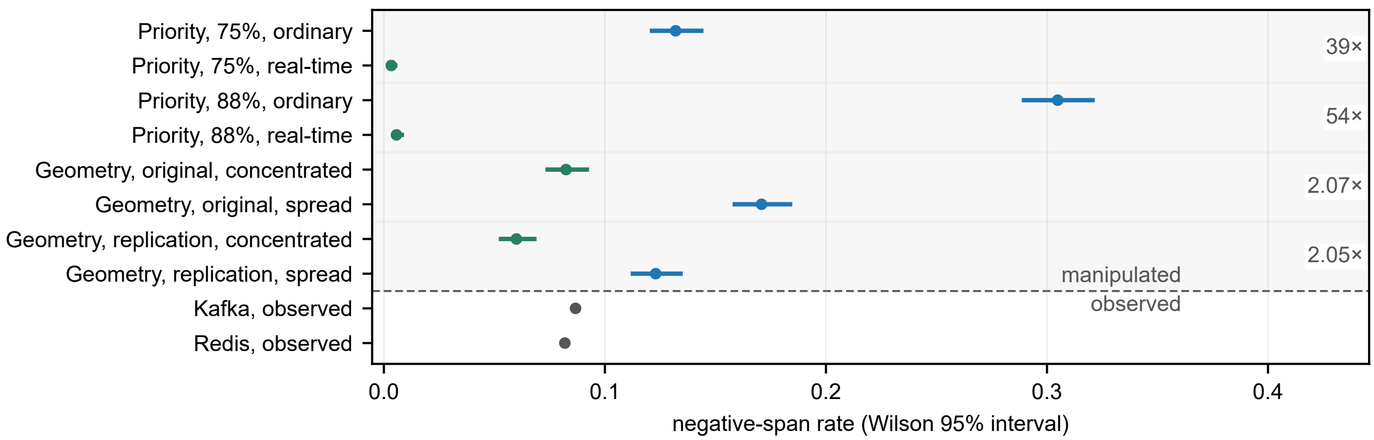

Fig. 6. **Every manipulation separates; the broker does not.** Wilson 95% intervals. Above the rule, the four matched pairs of Table II, utilization held fixed within each: what moves the rate is where the stamping thread sits, not how loaded the machine is. Below it, observed rather than manipulated, the two brokers of Table I; their intervals are narrower than the plotted point.

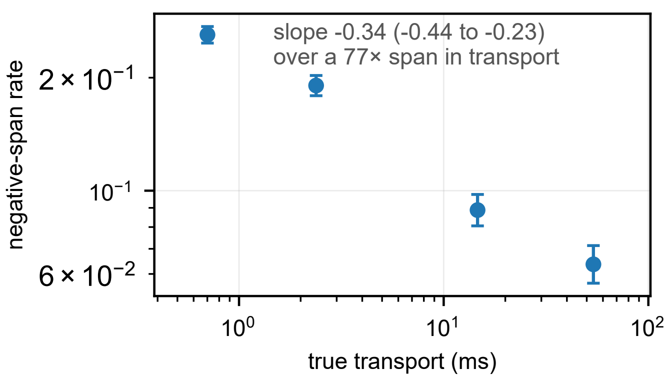

Fig. 7. **A longer interval inverts less often.** Negative span rate against true transport over the payload sweep, Wilson 95% intervals. The same stall distribution overlaps a short interval almost entirely and a long one hardly at all, which is Equation 3 as an experiment: lengthening what is being measured lowers the rate, where a load-driven account predicts no change.

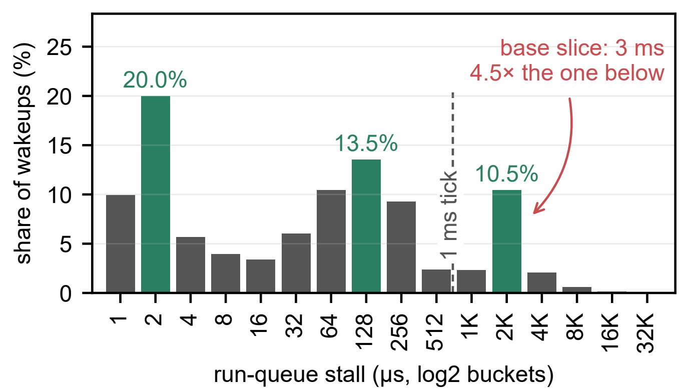

Fig. 8. **The traced run-queue stall distribution is trimodal.** 551,956 wakeups, `bpftrace` $\log_2$ buckets, modes shaded. The jitter core sits at a few microseconds and a second mode at 128–256 μs; the third, holding 10.5% of all wakeups, lands on the scheduler's derived base slice of 3 ms. A power law would have to descend monotonically across this axis.

The Pollaczek–Khinchine form remains motivation rather than fitted result.

D. The stall spectrum, and the scheduler's slice

That this distribution is not a single heavy tail is known — Gregg reported it as bimodal in the post introducing `runqlat`, the tool we use [38]. What we add is which modes, and why the upper one sits where it does.

Raising the payload from 0 to 256 KB lengthens the true transport 77×, and over that range the negative-span rate falls monotonically with T_{true} (Fig. 7); log–log least squares over the four payload levels returns a slope of -0.34 (-0.44 to -0.23, Student-t at two degrees of freedom), the same signed number the figure prints. The fit is in Supplement S32; four points do not earn an equation here. An earlier version read the kernel trace as an independent confirmation of it; we withdraw the claim, because estimating the traced index properly is what refuted it.

Over the 551,956 traced wakeups the grouped maximum-likelihood index on 0.25–2 ms is 1.19 (1.18–1.20) against an exceedance index on 0.5–2 ms of 0.21 (0.20–0.21). Two estimators of one quantity cannot differ sixfold unless the model is wrong, and a bootstrap goodness-of-fit agrees: $p < 0.0004$ over 2,500 replicates drawn from the fitted power law itself. The reason needs no fitting at all. The counts do not descend monotonically, as a power law must: they have 3 local maxima, the last a *mode* at 2–4 ms holding 10.5% of all wakeups, 4.5× its lower neighbor, above which the survival is light — grouped maximum likelihood beyond 4 ms gives 2.04 (2.00–2.07), a finite variance. Neither that share nor that position has, to our knowledge, been attributed to a scheduler constant before. Figure 8 is the histogram, and the shape is the argument: no monotone curve passes through it.

That mode is the preempted state of Equation 6, at a scale the maintainers had already named: with run-to-parity a task's lag is held within "$[-(\text{slice} + \text{tick}) : (\text{slice} + \text{tick})]$" [39]. The histogram adds that this is not merely a worst case but where a tenth of all wakeups land. The slice is *derived* rather than measured — the kernel's published rule [40] gives $4 \times 0.75\,\text{ms} = 3$ ms on the 8 cores the campaign's own load geometry recovers, over a 1 ms tick, and no scheduler tunable is written anywhere in the archived campaign (Supplement S44). A thread descheduled at the wrong instant therefore reads its clock about a slice late, which on a path whose true transport is 0.1–0.5 ms is six to thirty times the interval being measured. The tick is why the mode is a band rather than a spike. A mean over this distribution is dominated by a mode the operator never sees, so a mean-based counter cannot flag this failure.

VI. Discussion

Both failures are cheap to detect and neither is detected, so the remedy is a reporting rule rather than a better clock.

A. What the brokers actually do

Once the failed runs are removed, the broker comparison that motivated the work is small and one configuration choice dwarfs it. On gated artifacts the two sit within a millisecond on the transport proxy (TOST against a 1 ms margin, $p < 0.001$, across 3 levels), so it is not a purchasing argument; the one condition that reverses the ordering is a consumption pattern rather than the broker, and is invisible on a co-located testbed. The standard evaluation environment hides the choice that decides the outcome (Supplement S48).

B. For benchmark authors

SPEC and TPC judge a benchmark on five criteria [3], and the instrument we audit passes the half of *reproducibility* a reader checks — run-to-run consistency at a fixed configuration — *because* it fails *verifiability*: a median from 0.36% of the samples is the more stable for having had the variation deleted, and a second tester who picks a different send rate measures a different sample. Every rule below is an instance of the criterion that can see this.

Report a physical-consistency audit. Any cross-process timestamp difference admits a sign check that costs nothing, and every run it rejected here had passed every conventional check we knew. Better instruments exist [12], [13], [14]; whatever the instrument, check its output against causality and publish the rejection rate, as network clocks have for thirty years [17], [1]. The discipline is tested only by a campaign the rate costs something, and ours cost us a result.

Name the span you are subtracting. A difference of two stamps is a latency only if the first event causally precedes the second, and an acknowledgment at the producer does not precede a record at the consumer (Section III-A). We caught our own instance only on counting the send-referenced span separately (Table I).

Count your events before you quote a percentile. A median over as few as seven events per run is not a property of the system, and repetition will not tell you so.

Make the timestamping path unpreemptable, and say that you did. The one mitigation our measurements support directly: real-time priority on the stamping threads cut the negative-span rate 7–80× at unchanged background load, and busy-polling a dedicated core should buy the same for a burned core. Two caveats: the floor is not zero, so the check stays; and a real-time or spinning stamping thread changes the system it measures, so where that is unacceptable, report retention instead.

Dither the send instant, and publish the retained fraction. Randomizing probe timing to avoid synchronizing with a periodic phenomenon is long-standing advice [21], [22]; what we add is why — the phenomenon synchronized with is the timestamp quantum itself. Publish beside the latency the retention rate, the timestamp resolution and the synchronization state, none recoverable afterwards: a summary from 0.36% of samples is typographically identical to one from all. Neither half of the rule is ours; both are the detail beyond the metric that verifiability asks for, and the measurement standards agree — OWAMP requires a packet stamped in the future to be recorded unaltered within its timeout window [41], ETSI requires removed measurement cycles to be reported [42], and RFC 7679 treats negative singletons as usable distribution data [43]. The sign is the channel: negative means the reference stamp failed, computes-to-zero means the resolution failed, and one unsigned total cannot distinguish them. Supplements S39 and S40 give both precedents in full.

Record the achieved rate, not the flag meant to produce it. Verify it per run against elapsed wall time: the send schedule is an experimental variable in Mode B, not a nuisance parameter, and we found runs where configured and achieved rates disagreed.

None of this is hypothetical. A 2026 framework for the same comparisons already stamps in nanoseconds and subtracts a span that cannot invert, and reports sub-millisecond medians without needing a remedy [44] — both choices made, neither argued for (Supplement S52.2 reads its measurement section).

C. The limits of a better clock

Hardware-timestamped PTP [11] would take two LAN hosts from our $\approx 70\,\mu$s of `chrony` agreement to sub-microsecond, but behind a one-millisecond stamp, 70 μs and 70 ns produce byte-identical records: the guard deletes the same samples and the grid law is unchanged. The remedies are therefore ordered: record at native resolution, count what is discarded, dither the send instant, and only then is synchronization quality the frontier — our cross-host retention wandered 13.4–27.0% under clocks agreeing to well under a tick (Section IV-D). The obvious redesign, a consumer-to-producer acknowledgment, removes skew but not this failure, and every scheme that avoids the one-way requirement rests on an assumption its users rarely state (Supplements S37 and S43.6).

a) Resolution, as distinct from synchronization: Finer stamps fix the lesser half: a microsecond stamp retires Equation 4's deletions here and moves the grid to faster paths. The guard survives it and genuine negatives do too, so a better clock alone converts an uncounted *resolution* failure into an uncounted *instrument* one.

D. Threats and limitations

A negative span proves something is wrong; attributing it to scheduling is an inference, separated from its rivals by manipulation, with the eliminations listed at the end of Section V. Beyond that list, the clustering of consecutive late wakes ($z = -6.9$ at the co-located floor, at every load including none) is the signature of a shared scheduling delay, not independent kernel granularity. `chrony` reports the inter-host offset of Section III-A as comfortably sub-tick, but bounds its own error at 4–7 ms per host — the two worst of those bounds sum to 12 ms against a 1 ms stamp — so the cross-host channel stays open rather than ruled out. It did not produce the Redis-driver negatives, which the driver's own stamp accounts for exactly (Section IV-D), but it is why we claim no cross-host bound in general.

a) Scope, and the alternatives eliminated: Three widely used runtimes document this structure without drawing its consequence for a cross-process subtraction (Supplement S38). The mechanism requires a stamping path that can be preempted. Designs that pin a thread to a dedicated core and busy-poll, or that stamp in hardware, remove the state Equation 6 depends on, and we make no claim about them. Within our own setting the alternatives are each eliminated: clock skew (one clock by construction on the rejecting arms; zero negatives in $905{,}040$ two-clock samples); cross-CPU incoherence under migration, below; utilization (identical ρ, rates twofold apart); timer noise (`SCHED_FIFO` at fixed utilization collapses the rate 7–$80\times$ across 8 pairs, and noise does not respect scheduling class); stress (transport $77\times$ longer *lowers* the rate); and direct observation (a `sched_switch` histogram predicts the rate to within a third, unfitted). One rival is not the scheduler's: the stamp is a CPython call, so it waits on the interpreter lock too, invisibly to a tracepoint probe — bounded because that kernel-only estimator brackets the observed rate $(0.78, 1.06, 1.32)$ instead of under-predicting (Supplement S43.4).

Clock incoherence across vCPUs needs its own answer, our machines being virtual [45], and gets one twice over: the smallest increment the probe saw, 90 ns, admits only `kvm-clock` or `tsc`, both of which the kernel uses solely where cross-CPU coherence is asserted, and the corpus closes the channel again on headroom, not on shared endpoints — the send-referenced span holds a floor $455\times$ too high to explain the deepest negative span (Supplement S42).

The mechanism's constants are those of one EEVDF-era kernel (6.8); CFS-era schedulers, the regime Lozi et al. document, may shift them. The deletion law is demonstrated on one benchmark and reproduced in one independent harness, and the five tools of Section IV-D are readings of source rather than measured deployments. Nor is the guard confined to the tool we measured: 40 of the 40 public forks we could read carry the expression unchanged, surveyed 2026-08-24. Its distributed mode failed on every attempt, so that tool's cross-host exposure rests on source reading and our own two-host harness (Supplement S4). Exposure depends on a deployment's path latency and producer rate, which is why we recommend publishing a retention rate rather than calling any published number wrong.

VII. Conclusion

A latency benchmark measures the system only while the instrument is faster than the thing being measured. Below that line the two failures reported here appear, invisible in the output: two stamps written by threads that wait for a core independently invert the interval between them whenever those waits differ by more than it, and a stamp quantized to a millisecond, filtered for positivity, deletes most of a sub-millisecond path by arithmetic the operator never sees. Neither needs a better clock, and neither is expensive to detect. Check the spans whose endpoints are stamped in different places, count what your instrument throws away, and publish the retention rate beside the number.

Acknowledgment

The author thanks R. Duvignau, whose reading established that the acknowledgment-referenced span is not a causal chain and prompted the recount that reorganized this paper; D. Gregg, for the readability critique that shaped the previous version and for the self-consistency argument in Section IV; J. Kunkel, for the same-clock protocol contrast, the offset-estimation route and the measurement-model figure; N. Herbst, for load-generation validation and the timer-characterization literature; M. Luthra, for the kernel-tracing pointers; M. Kogias, for the probe-placement argument; A. Ousterhout, for the preemptibility scope condition; and L. Tratt, for the question about timestamp resolution answered in Section IV.

In accordance with IEEE policy, Anthropic's Claude assisted with code development and verification, manuscript editing, and literature search. Experimental design, scientific claims, and interpretation are the author's, and all reported quantities are reproducibly computed from the committed data and archived code.

Artifact Availability

Code, analysis and manuscript are archived at 10.5281/zenodo.21650031, the measurement dataset at 10.5281/zenodo.21650064; both resolve to the current version, `v2.6.0`. Every number here is recomputed from the committed data by the archived code at build time, and the test suite fails if the text and the data disagree.

References

[1] V. Paxson, "Strategies for sound Internet measurement," in *Proc. ACM Internet Measurement Conf. (IMC)*. ACM, 2004, pp. 263–271, physical impossibility as grounds for discarding a trace.

[2] Standard Performance Evaluation Corporation, "SPECjbb2015 run and reporting rules, v1.02," 2018, §3.4.5; an initial clock offset beyond ten minutes voids the run.

[3] J. von Kistowski, J. A. Arnold, K. Huppler, K.-D. Lange, J. L. Henning, and P. Cao, "How to build a benchmark," in *Proc. 6th ACM/SPEC Int. Conf. on Performance Engineering (ICPE)*, 2015, pp. 333–336, section 3, the five quality criteria of a standardized benchmark.

[4] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/; accessed 19 August 2026.

[5] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2009, pp. 265–276.

[6] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. ACM, 2007, pp. 57–76.

[7] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2015, pp. 73:1–73:12.
[8] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *Proc. IEEE 34th Int. Conf. Data Engineering (ICDE)*, 2018, pp. 1507–1518.
[9] G. Van Dongen and D. Van den Poel, "Evaluation of stream processing frameworks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 8, pp. 1845–1858, 2020.
[10] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, 1978.
[11] IEEE, "IEEE standard for a precision clock synchronization protocol for networked measurement and control systems," IEEE Std 1588-2019, 2020, revision of IEEE Std 1588-2008.
[12] S. Yang, J. Jeong, B. Scholz, and B. Burgstaller, "Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads," *arXiv preprint arXiv:2205.09325*, 2022.
[13] M. Kogias, S. Mallon, and E. Bugnion, "Lancet: A self-correcting latency measuring tool," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2019, pp. 881–896, §4.3: terminates and reports the results with an inconclusive verdict.
[14] R. Kundel, "P4STA: High precision network function benchmarking," in *Accelerating Network Functions Using Reconfigurable Hardware*, ser. Springer Theses. Springer Nature Switzerland, 2024, pp. 93–115.
[15] M. Kuperberg, M. Krogmann, and R. H. Reussner, "Metric-based selection of timer methods for accurate measurements," in *Proc. 2nd ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, Mar. 2011, pp. 151–156.
[16] B. Villain, M. Davis, J. Ridoux, D. Veitch, and N. Normand, "Probing the latencies of software timestamping," in *Proc. IEEE Int. Symp. Precision Clock Synchronization for Measurement, Control and Communication (ISPCS)*, 2012, pp. 1–6, dTrace breakdown against a DAG hardware reference.
[17] V. Paxson, "On calibrating measurements of packet transit times," in *Proc. ACM SIGMETRICS*. ACM, 1998, pp. 11–21.
[18] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, "Time, causality, and observability failures in distributed AI inference systems," *arXiv preprint arXiv:2604.21361*, Apr. 2026, open Compute Project, Unified Intelligent Infrastructure workstream.
[19] OpenMessaging Benchmark contributors, "Can the average publish latency be larger than end-to-end latency? (issue #216)," https://github.com/openmessaging/benchmark/issues/216, 2021, the anomaly attributed to inter-node skew, on a same-node report.
[20] J. C. Gust, R. M. Graham, and M. A. Lombardi, "Stopwatch and timer calibrations (2009 edition)," National Institute of Standards and Technology, NIST Special Publication 960-12, 2009, uncertainty budget carrying both operator reaction time and display resolution.
[21] V. Paxson, G. Almes, J. Mahdavi, and M. Mathis, "Framework for IP performance metrics," Internet Engineering Task Force, RFC 2330, May 1998.
[22] V. Raisanen, G. Grotefeld, and A. Morton, "Network performance measurement with periodic streams," Internet Engineering Task Force, RFC 3432, Nov. 2002.
[23] G. Tene, "How NOT to measure latency," 2015, talk, Strange Loop / QCon; coined "coordinated omission".
[24] M. Wiederhold, "Sub-millisecond latencies are reported as taking 0 milliseconds (issue #41)," Yahoo! Cloud Serving Benchmark, Oct. 2011, https://github.com/brianfrankcooper/YCSB/issues/41; fixed by pull request #42 and released in 0.1.4.
[25] C. Millsap and J. Holt, *Optimizing Oracle Performance*. Sebastopol, CA: O'Reilly, 2003, section 7.7, "Quantization Error".
[26] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766; undated; the year is its *HP Journal* citation.
[27] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, "The Linux scheduler: a decade of wasted cores," in *Proc. 11th European Conf. Computer Systems (EuroSys)*. ACM, 2016, pp. 1–16.
[28] A. Chandrasekar and J. Kramberger, "Identifying and mitigating systemic measurement bias in production LLM inference benchmarks," *arXiv preprint arXiv:2605.24217v2*, 2026.
[29] S. Bradner, "Benchmarking terminology for network interconnection devices," Internet Engineering Task Force, RFC 1242, 1991, section 3.9 defines latency for cut-through devices, where the value can be negative because forwarding begins before reception completes.
[30] Y. Virkar and A. Clauset, "Power-law distributions in binned empirical data," *The Annals of Applied Statistics*, vol. 8, no. 1, pp. 89–119, 2014.
[31] S. Guo, "Avoid adding negative metric values (pull request #56)," OpenMessaging Benchmark, commit ebb5d6b8, Mar. 2018, https://github.com/openmessaging/benchmark/pull/56; accessed 18 August 2026.
[32] Apache Pulsar contributors, "`PerformanceConsumer.java`," Source repository, 2026, https://github.com/apache/pulsar. Read 2026-08-21.
[33] J. Axboe, "fio: Flexible I/O tester — `gettime.c`," Source repository, 2026, https://github.com/axboe/fio; the guard is in three timing functions. Read 2026-08-20.
[34] A. D. Brunelle, "`blktrace/btt — btt/inlines.h`," Source repository, 2026, `tdelta()` returns one nanosecond when the stage timestamps are out of order: `return (from < to) ? (to - from) : 1;`. Read 2026-08-20.
[35] IOP Systems, "Rezolus: systems performance telemetry — `scheduler_discarded_samples`," Source repository, 2026, https://github.com/iopsystems/rezolus; counts samples discarded for out-of-order timestamps. Read 2026-08-20.
[36] EMQ Technologies, "`emqtt-bench`: MQTT benchmark tool — `src/emqtt_bench.erl`," Source repository, 2026, https://github.com/emqx/emqtt-bench; the guard is on the counter, not the sample. Read 2026-08-21.
[37] Apache Kafka contributors, "KAFKA-19888: Clamp negative values in coordinator histograms," Issue tracker and pull request #20986, 2025, https://issues.apache.org/jira/browse/KAFKA-19888; merged 2025-11-26, shipped in 4.2.0, backported to 4.0.2/4.1.2. Read 2026-08-21.
[38] B. Gregg, "Linux bcc/BPF run queue (scheduler) latency," https://www.brendangregg.com/blog/2016-10-08/linux-bcc-runqlat.html, Oct. 2016, introduces `runqlat`; reports the distribution as bimodal without attributing a mechanism.
[39] V. Guittot, "`sched/fair`: Manage lag and run to parity with different slices," Linux kernel mailing list, Jul. 2025, states the base-slice protection and the slice-plus-tick lag bound.
[40] Z. Zhou, "sched: Reduce the default slice to avoid tasks getting an extra tick," Linux kernel mailing list, commit 2ae891b82695, 2025, reviewed-by V. Guittot; reduces the constant to `0.70` in v6.15.
[41] S. Shalunov, B. Teitelbaum, A. Karp, J. W. Boote, and M. J. Zekauskas, "A one-way active measurement protocol (OWAMP)," Internet Engineering Task Force, RFC 4656, Sep. 2006.
[42] ETSI, "Speech and multimedia transmission quality (STQ); QoS aspects for popular services in mobile networks; part 4: Requirements for quality of service measurement equipment," European Telecommunications Standards Institute, Technical Specification TS 102 250-4 V2.3.1, Nov. 2019, clause 5.1, General requirement for data logging.
[43] G. Almes, S. Kalidindi, M. Zekauskas, and A. Morton, "A one-way delay metric for IPPM," Internet Engineering Task Force, RFC 7679, Jan. 2016, obsoletes RFC 2679. Section 3.5 keeps negative values as distribution data rather than discarding them.
[44] T. C. Paul, P. Lertpongrujikorn, H. D. Nguyen, and M. Amini Salehi, "Benchmarking message brokers for IoT edge computing: A comprehensive performance study," arXiv preprint arXiv:2603.21600, 2026, introduces mq-bench: nanosecond send stamps, one host, no positivity guard. Read 2026-08-25.
[45] Linux Kernel Documentation, "KVM-specific MSRs: `MSR_KVM_SYSTEM_TIME_NEW`," https://docs.kernel.org/virt/kvm/x86/msr.html, 2026, flag bit 0, CPUID `0x40000001` bit 24. Read 2026-08-21.

Gustavo Pedro Ricou received the B.Sc. degree in physics from the Universidade do Porto, Portugal, and specialized in nuclear fusion science at the University of Stuttgart and the Université de Lorraine. He holds the Higher Diploma in computing from Atlantic Technological University and an M.Sc. in software design with cloud-native computing from the Technological University of the Shannon, both with first-class honors, and is currently pursuing an M.Sc. in statistics and data science at Trinity College Dublin, Ireland. He is a Software Engineer with BoscaSports, building real-time pipelines for live sports. His research concerns measurement validity — what a reported number is really evidence of — in computer systems, environmental measurement and sports analytics, using pre-registration and physical-consistency auditing.